

Written Report – 6.419x Module 2

Name: (Vikhyaat)

▪ Problem 2

Part 1 : Visualization

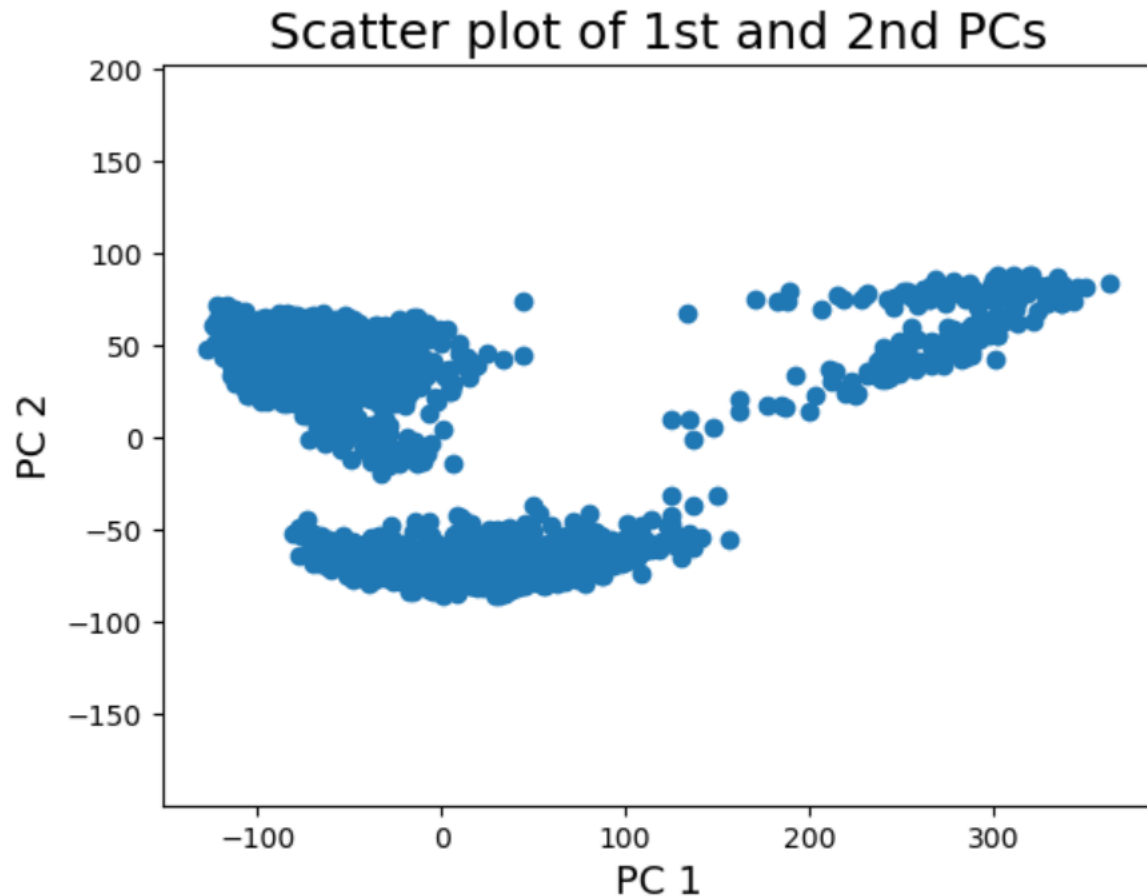
1(3 points) Provide at least one visualization which clearly shows the existence of three main brain cell types as described by the scientist, and explain how it shows this. Your visualization should support the idea that cells from different groups can differ greatly. (We recommend ~50-100 words. Maximum 200 words)

Solution: The provided scatter plot of the 1st and 2nd Principal Components (PCs) serves as a visualization demonstrating the existence of three main brain cell types.

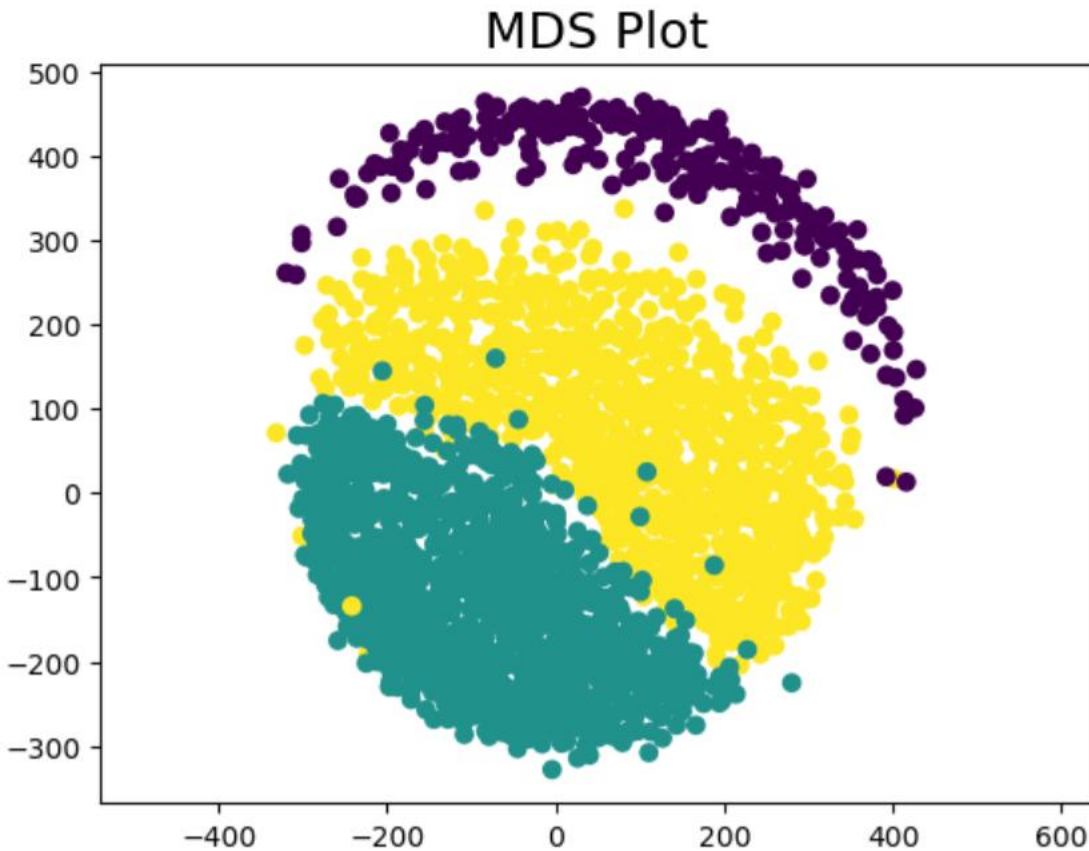
This visualization clearly shows three distinct, spatially separated clusters of points:

1. A prominent cluster on the left side of the plot.
2. A distinct, more compact cluster situated below the first.
3. A third, separate cluster positioned towards the right of the plot.

These groupings, or "clusters," represent cells with similar underlying characteristics (e.g., gene expression profiles) projected into a lower-dimensional space by PCA.



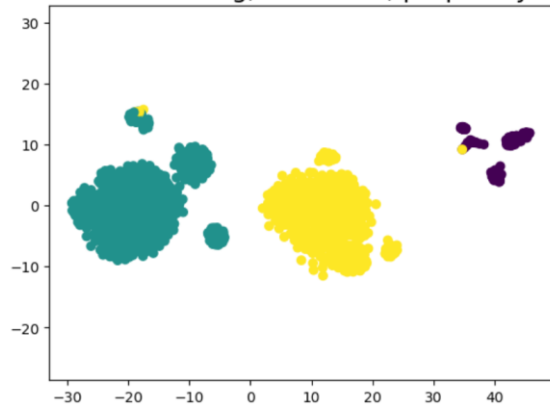
The Multidimensional Scaling (MDS) when appropriately analyzed with clustering information and coloring the points accordingly, three distinct clusters of points emerged.



2. (4 points) Provide at least one visualization which supports the claim that within each of the three types, there are numerous possible sub-types for a cell. In your visualization, highlight which of the three main types these sub-types belong to. Again, explain how your visualization supports the claim. (We recommend ~100 words. Maximum 200 words)

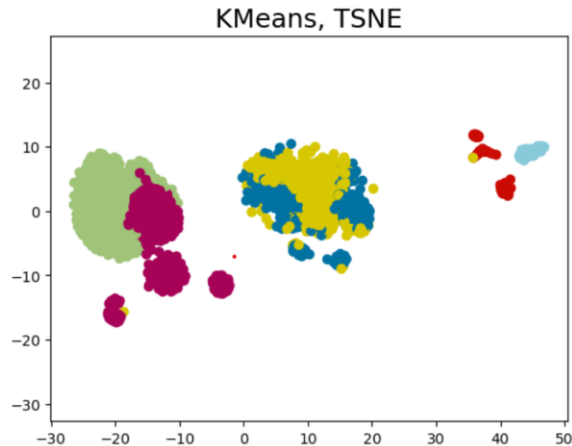
Solution:

KMeans Clustering, TSNE Plot, perplexity =40



t-SNE plot with KMeans clustering $K=3$ does support the claim that within each of the three main types, there are possible sub-types for a cell.

With $K=6$ and tsne plot we clearly are able to show cell subtypes as tsne keeps local neighbours close together and preserves local structure.



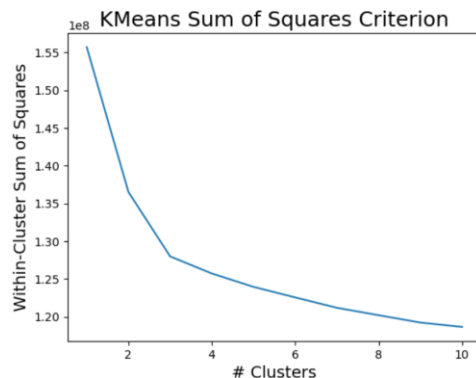
Part 2 Unsupervised Feature Selection

1. (4 points) Using your clustering method(s) of choice, find a suitable clustering for the cells. Briefly explain how you chose the number of clusters by appropriate visualizations and/or numerical findings. (to cluster cells into the subcategories instead of categories) (We recommend ~150 words. Maximum 250 words)

Solution:

The provided image displays an **Elbow Plot** used in K-Means clustering to determine the optimal number of clusters, often represented by k . The Y-axis represents the "Within-Cluster Sum of Squares" (WCSS) or inertia, which measures the compactness of the clusters, while the X-axis represents the number of clusters. The principle of the elbow method is to identify the point on the curve where the rate of decrease in WCSS significantly slows down, forming an "elbow" shape.

The most significant "elbow" in the curve appears to be around **3 clusters**. However, there is scope for improvement of WCSS beyond 3 clusters and we test which clusters > 3 produces appropriate subtypes within three main clusters based on "Average Silhouette Scores,"

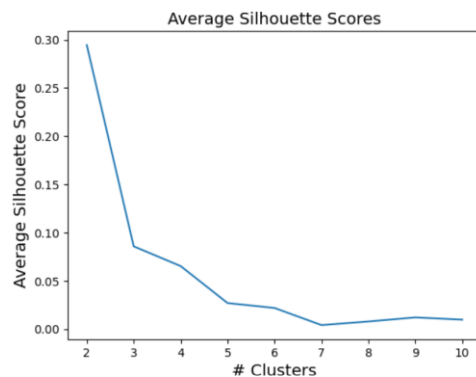


"Average Silhouette Scores," is a crucial visualization for determining the optimal number of clusters in a dataset using the **Silhouette Method**.

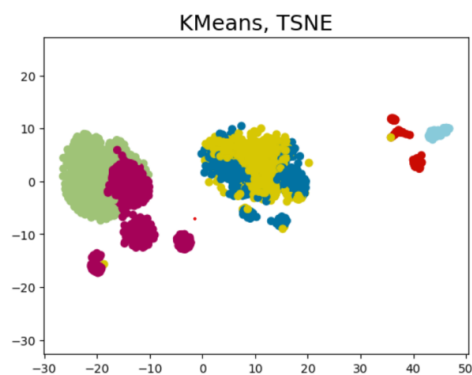
The Silhouette Score for a data point ranges from -1 to +1.

- A high positive value (close to 1) indicates that the data point is well-matched to its own cluster and well-separated from neighboring clusters, implying a strong clustering configuration.
- A value near 0 suggests that the data point is close to the decision boundary between two clusters, indicating potential overlap or ambiguity in assignment.

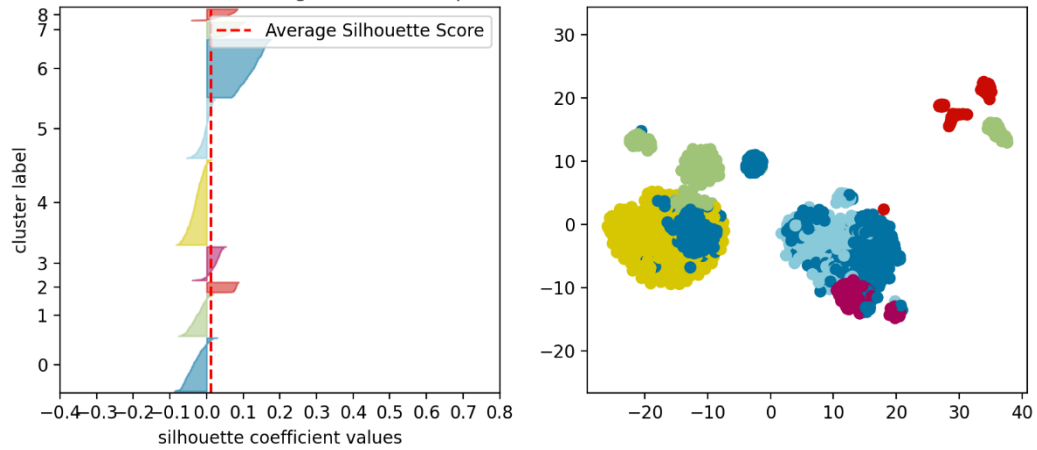
- Observation from the plot:** In the plot, the Average Silhouette Score is highest when the number of clusters is **2**, at approximately 0.29. As the number of clusters increases, the average silhouette score steadily declines, reaching very low values for 7 or more clusters. For 7 or more clusters there is misassignment of cells from one main cluster to other as expected from low Average Silhouette Score. Similarly, for cluster 3-5 there are no subtypes within each of the main clusters. Cluster 6 shows subtypes within each main cluster and is the most appropriate choice.



With K=6 and tsne plot we clearly are able to show cell subtypes as tsne keeps local neighbours close together and preserves local structure.



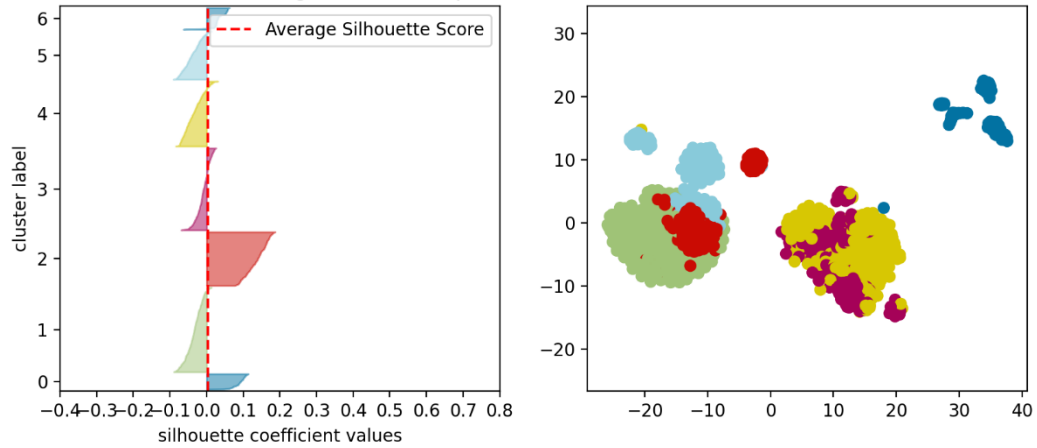
Silhouette Plot of KMeans Clustering for 2169 Samples in 9 Centers



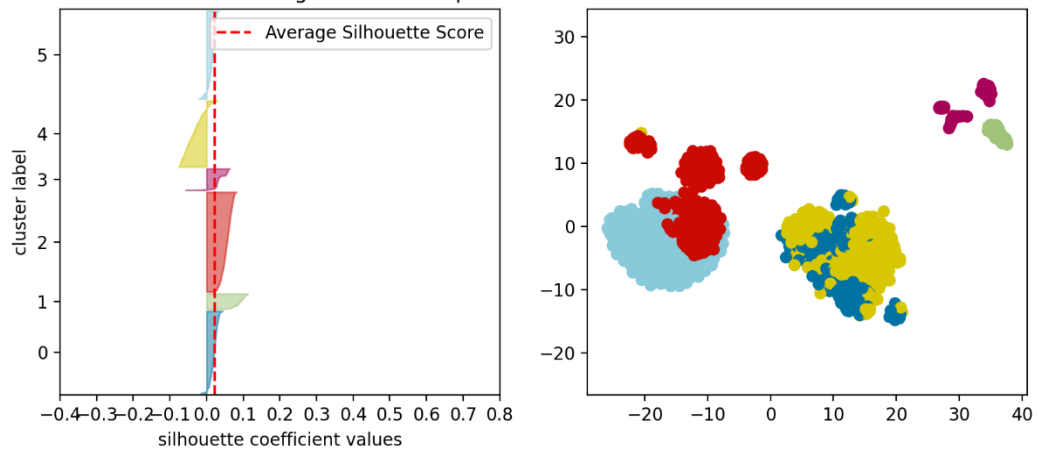
Silhouette Plot of KMeans Clustering for 2169 Samples in 8 Centers



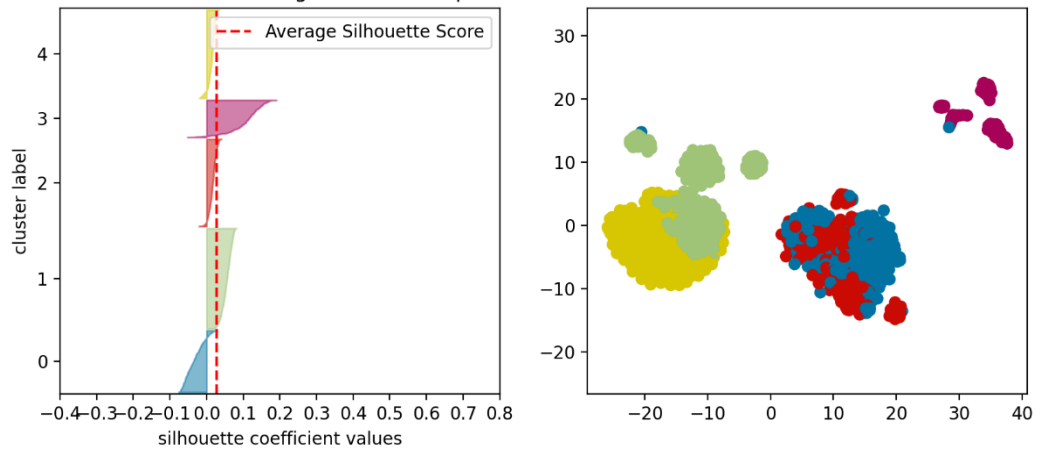
Silhouette Plot of KMeans Clustering for 2169 Samples in 7 Centers



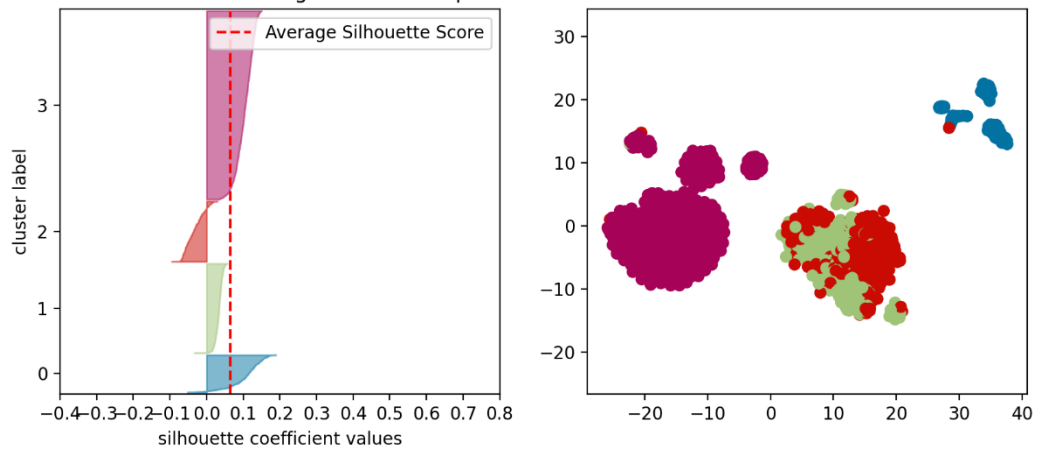
Silhouette Plot of KMeans Clustering for 2169 Samples in 6 Centers



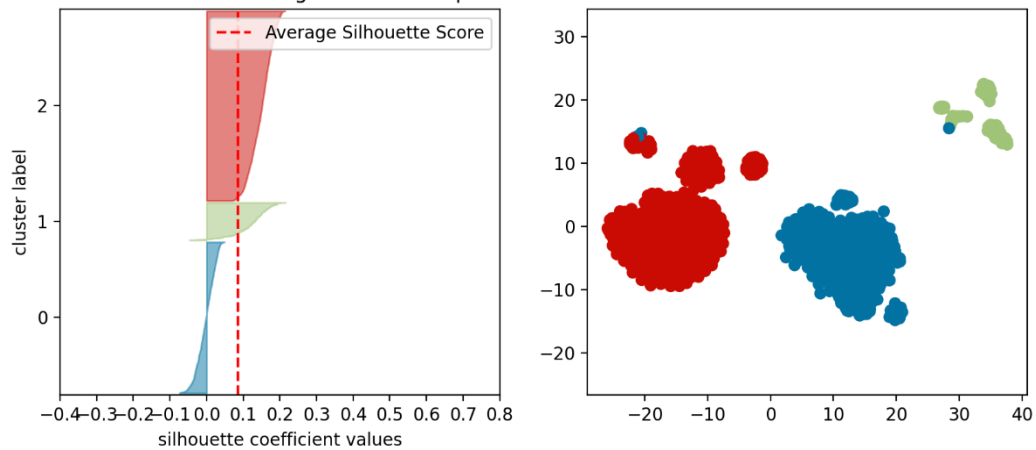
Silhouette Plot of KMeans Clustering for 2169 Samples in 5 Centers



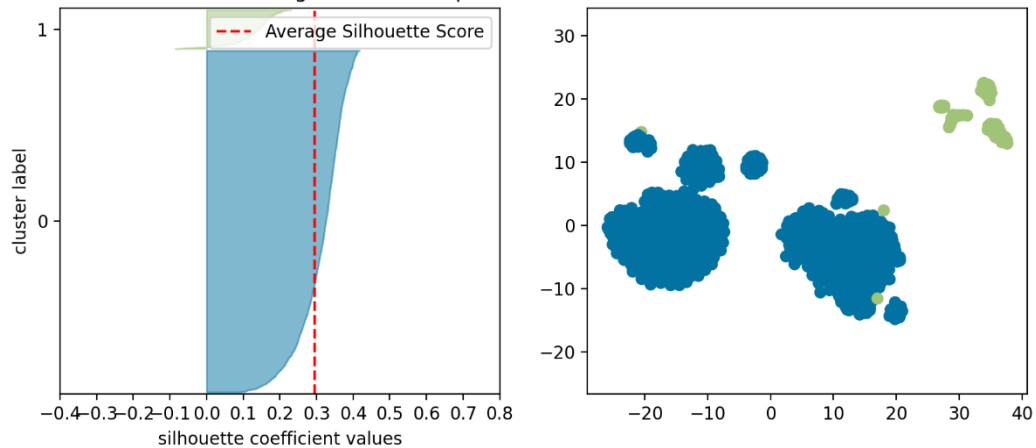
Silhouette Plot of KMeans Clustering for 2169 Samples in 4 Centers



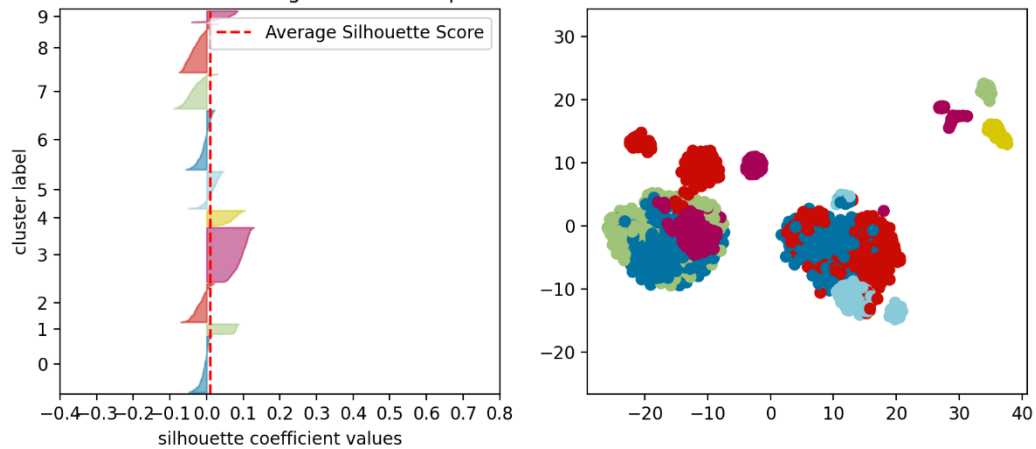
Silhouette Plot of KMeans Clustering for 2169 Samples in 3 Centers



Silhouette Plot of KMeans Clustering for 2169 Samples in 2 Centers



Silhouette Plot of KMeans Clustering for 2169 Samples in 10 Centers



2.(6 points) We will now treat your cluster assignments as labels for supervised learning. Fit a logistic regression model to the original data (not principal components), with your clustering as the target labels. Since the data is high-dimensional, make sure to regularize your model using your choice of , , or elastic

net, and separate the data into training and validation or use cross-validation to select your model. Report your choice of regularization parameter and validation performance. (We recommend ~100-150 words. Maximum 250 words)

Solution: A logistic regression model was fit by first splitting the data into training and testing. 4/5th of the data was used for training purposes. Using cluster assignments as target labels and without L1 regularization, a logistic regression gave an accuracy score of 0.9907834101382489.

```
log_reg = LogisticRegression(penalty=None,multi_class="ovr").fit(X_train,y_train)
```

With L1 regularization parameter C= 0.1, liblinear solver and max_iter=5000 accuracy score decreases to 97%

```
log_reg=LogisticRegression(penalty="l1",C=0.1,solver="liblinear",max_iter=5000,multi_class="ovr").fit(X_train,y_train)
```

Using LogisticRegressionCV which incorporates built-in cross-validation for hyperparameter tuning. Instead of you manually searching for the best C value, LogisticRegressionCV performs this search internally. It takes a range of C values (either explicitly provided or generated automatically) and uses cross-validation to evaluate the performance of LogisticRegression for each C value. It then selects C for each class that yields the best cross-validation score.

```
log_reg=LogisticRegressionCV(cv=5,Cs=[0.01,0.1,1,10],max_iter=5000,penalty="l1",solver="liblinear",multi_class="ovr")
```

Optimal regularization parameter (C) for each class were determined to be array([0.01, 1. , 10. , 0.1 , 10. , 0.01]). This model achieved a validation accuracy of 97%, similar to LogisticRegression with C= 0.1, a relatively strong regularization, which shrinks the model coefficients to zero and prevent overfitting.

3.(9 points) *Select the features with the top 100 corresponding coefficient values (since this is a multi-class model, you can rank the coefficients using the maximum absolute value over classes, or the sum of absolute values). Take the evaluation training data in p2_evaluation and use a subset of the genes consisting of the features you selected. Train a logistic regression classifier on this training data, and evaluate its performance on the evaluation test data. Report your score. (Don't forget to take the log transform before training and testing.)*

Compare the obtained score with two baselines: random features (take a random selection of 100 genes), and high-variance features (take the 100 genes with highest variance). Finally, compare the variances of the features you selected with the highest variance features by plotting a histogram of the variances of features selected by both methods. (We recommend ~100-150 words. Maximum 200 words)

Solution:

1. Data Loading and Preprocessing:

- Load the p2_evaluation training and test datasets.
- Apply a log transform (e.g., $\text{np.log2}(x + 1)$) to both training and test feature data.

```
X_train_log = np.log2(X_train_eval + 1)
```

```
X_test_log = np.log2(X_test_eval + 1)
```

2. Feature Selection based on Coefficients on already trained logistic regression model from previous problem :

- For each feature, calculate a single ranking score from its coefficients. This can be done by taking the sum of absolute values (`np.sum(np.abs(coefficients), axis=0)`)
- Select the top 100 features based on these ranking scores.

```
feature_scores=np.sum(np.abs(coefficients),axis=0)
indices_of_largest = np.argsort(feature_scores)[-100:]
```

3. Training and Evaluation with Coefficient-Selected Features:

- Subset the log-transformed training and test data to include only the selected 100 features.
`X_train_coef_selected = X_train_log[indices_of_largest]`
`X_test_coef_selected = X_test_log[indices_of_largest]`
- Train a new logistic regression classifier on this subsetted training data.
`log_reg=LogisticRegressionCV(cv=5,Cs=[0.01,0.1,1,10],max_iter=5000,penalty="l1",solver="liblinear",multi_class="ovr")`
`log_reg.fit(X_train_coef_selected,y_train_eval)`
- Evaluate the classifier's performance on the subsetted test data and record the score
`print(f"Score with coefficient-selected features: {log_reg.score(X_test_coef_selected,y_test_eval)}")`

Score with coefficient-selected features: 0.8474729241877257

4. Baseline 1: Random Features:

- Randomly select 100 features from the original dataset.
`np.random.seed(42) # for reproducibility`
`random_features = np.random.choice(X_train_log.columns, 100, replace=False)`
- Subset the log-transformed training and test data using these random features.
`X_train_random = X_train_log[random_features]`
`X_test_random = X_test_log[random_features]`
- Train a logistic regression classifier and evaluate its performance on the test data.
`log_reg_random=LogisticRegressionCV(cv=5,Cs=[0.01,0.1,1,10],max_iter=5000,penalty="l1",solver="liblinear",multi_class="ovr")`
`log_reg_random.fit(X_train_random, y_train_eval)`
`score_random = log_reg_random.score(X_test_random, y_test_eval)`
`print(f"Score with 100 random features: {score_random:.4f}")`

Score with 100 random features: 0.3439

5. Baseline 2: High-Variance Features:

- Calculate the variance of each feature in the log-transformed training data.
`feature_variances = X_train_log.var()`
- Select the 100 features with the highest variance.

```
top_100_variance_indices = np.argsort(feature_variances)[-100:]
```

```
high_variance_features = X_train_log.columns[top_100_variance_indices]
```

- Subset the log-transformed training and test data using these high-variance features.

```
X_train_variance = X_train_log[high_variance_features]
```

```
X_test_variance = X_test_log[high_variance_features]
```

- Train a logistic regression classifier and evaluate its performance on the test data. Record the score.

```
log_reg_variance=LogisticRegressionCV(cv=5,Cs=[0.01,0.1,1,10],max_iter=5000,penalty="l1",solver="liblinear",multi_class="ovr")
```

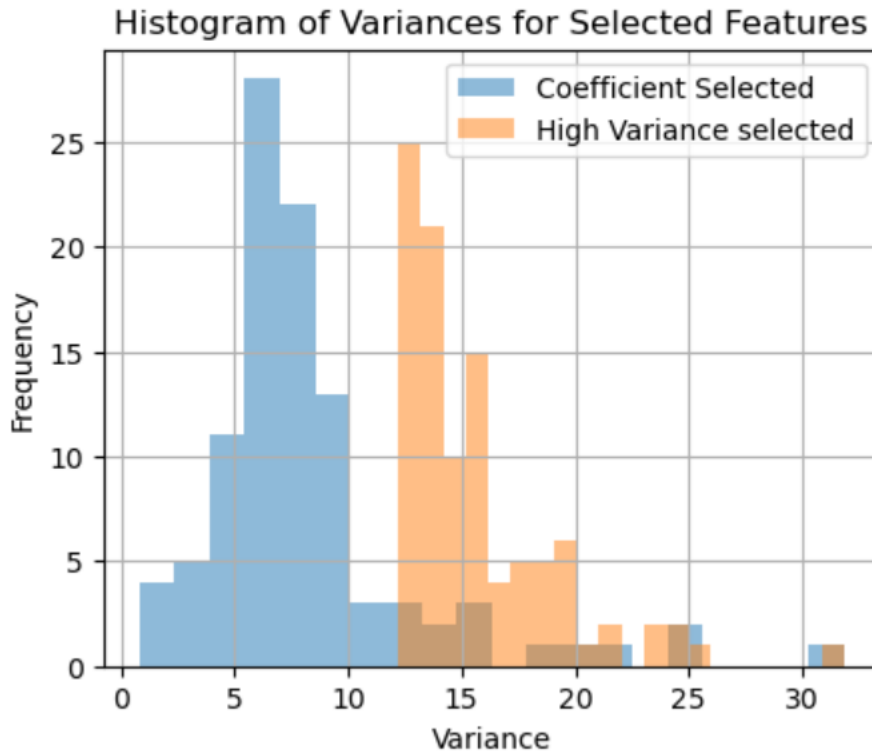
```
log_reg_variance.fit(X_train_variance, y_train_eval)
```

```
score_variance = log_reg_variance.score(X_test_variance, y_test_eval)
```

```
print(f"Score with top 100 high-variance features: {score_variance:.4f}")
```

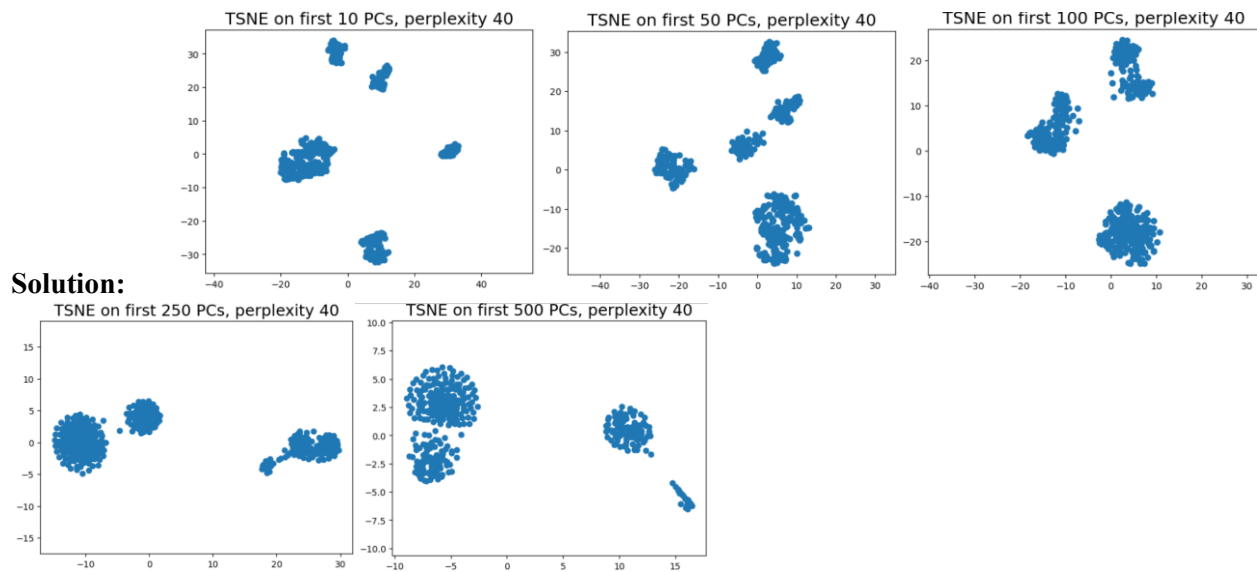
Score with top 100 high-variance features: 0.9143

Score of 100 high-variance features is comparable to 100 high coefficient selected features whereas 100 random features score is very low. This suggest that 100 highest coefficient selected features are those that have highest variance.:



■ Problem 3

1.(3 points) When we created the T-SNE plot in Problem 1, we ran T-SNE on the top 50 PC's of the data. But we could have easily chosen a different number of PC's to represent the data. Run T-SNE using 10, 50, 100, 250, and 500 PC's, and plot the resulting visualization for each. What do you observe as you increase the number of PC's used? (We recommend ~50 words. Maximum 150 words)



As no. of PC becomes larger than 50 it becomes difficult to clearly distinguish the clusters and clusters become more diffuse. This happens because t-SNE struggles to accommodate both the global structure

(from the top PCs) and the more complex local structure (from the smaller PCs). As PCs increase, we are capturing more noise which makes it difficult to clearly distinguish some clusters.

2.(13 points) *Pick three hyper-parameters below (the 3 is the total number that a report needs to analyze. It can take a) 2 from A, 1 from B, or b) 1 from A, 2 from B.) and analyze how changing the hyper-parameters affect the conclusions that can be drawn from the data. Please choose at least one hyper-parameter from each of the two categories (visualization and clustering/feature selection). At minimum, evaluate the hyper-parameters individually, but you may also evaluate how joint changes in the hyper-parameters affect the results. You may use any of the datasets we have given you in this project. For visualization hyper-parameters, you may find it productive to augment your analysis with experiments on synthetic data, though we request that you use real data in at least one demonstration. (We recommend ~250 words. Maximum 400 words)*

Some possible choices of hyper-parameters are:

Category A (visualization):

- *T-SNE perplexity*
- *T-SNE learning rate*
- *T-SNE early exaggeration*
- *T-SNE initialization*
- *T-SNE number of iterations/convergence tolerance*

Category B (clustering/feature selection):

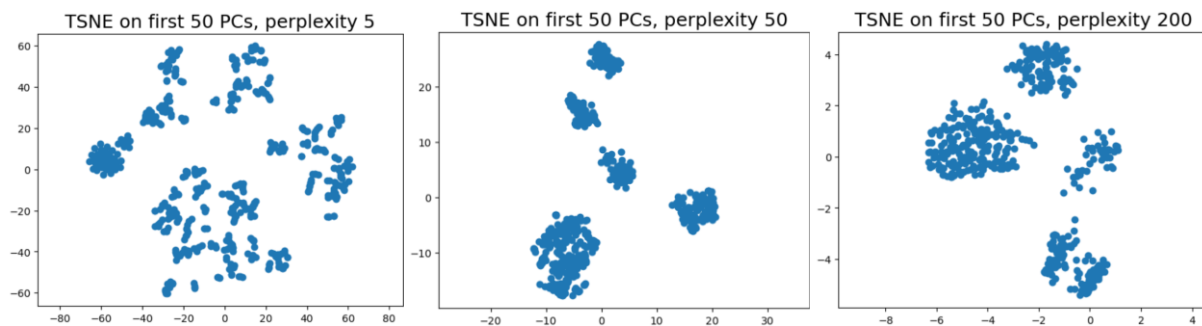
- *Effect of number of PC's chosen on clustering*

- *Type of clustering criterion used in hierarchical clustering (single linkage vs ward, for example)*
- *Number of clusters chosen for use in unsupervised feature selection and how it affects the quality of the chosen features*
- *Magnitude of regularization and its relation to your feature selection (for example, does under or over-regularizing the model lead to bad features being selected?)*
- *Type of regularization (, , elastic net) in the logistic regression step and how the resulting features selected differ*

For visualization hyper-parameters, provide substantial visualizations and explanation on how the parameter affects the image. For clustering/feature selection, provide visualizations and/or numerical results which demonstrate how different choices affect the downstream visualizations and feature selection quality. Provide adequate explanations in words for each of these visualizations and numerical results.

Solution:

- *T-SNE perplexity*



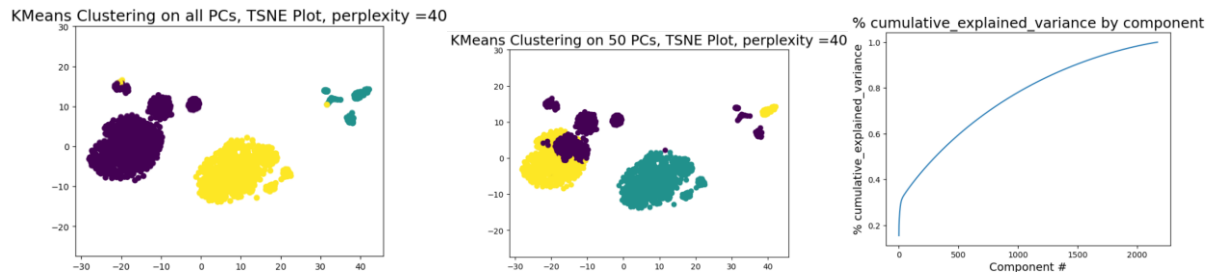
The perplexity parameter in t-SNE is a crucial tuning knob that controls the balance between preserving local and global structures in your data.

At low perplexity ~ 5 the focus is on preserving local structure where points that are very close to each other will form tight, isolated groups. The algorithm may mistakenly treat noisy local variations as significant structures, causing random data to appear clustered.

At high perplexity ~ 200 algorithm consider a larger neighborhood of points which helps reveal the overall, broader structure of the dataset. By taking a wider view, the algorithm may merge clusters that were distinct at lower perplexity

- *Effect of number of PC's chosen on clustering*

One needs to include sufficient of PCs to ensure that it explains 85 % of total variance in the data. Otherwise k mean clustering will not be able to distinguish desired no of clusters as shown below :



Restricting the no of clusters can help cut out the noise and help clear separation of clusters in some data sets. It will run the algorithm faster.

- *Magnitude of regularization and its relation to your feature selection (for example, does under or over-regularizing the model lead to bad features being selected?)*

Over-regularizing the model (Low C parameter))

```
log_reg=LogisticRegression(penalty="l1",C=0.01,solver="liblinear",max_iter=5000,multi_class="ovr").fit(X_train,y_train)
```

```
log_reg.score(X_train,y_train)
```

Output : 0.3732394366197183

```
log_reg.score(X_test,y_test)
```

Output: 0.16666666666666666

Effect: A low C value of 0.01 applies a strong penalty, which heavily constrains the model and forces it to be simpler. Resulting feature selection: With L1 regularization, a high penalty will force the coefficients of many features to become zero, potentially dropping relevant features along with the irrelevant ones.

Consequence: The model may be too simple to capture the underlying patterns in the data, leading to underfitting. This means the model will perform poorly on both the training and test data, as important signals were discarded during the feature selection. In this case, over-regularizing leads to "bad" feature selection because it eliminates good features.

Under-regularizing the model (High C parameter)

```
log_reg=LogisticRegression(penalty="l1",C=10 ,solver="liblinear",max_iter=5000,multi_class="ovr").fit(X_train,y_train)
```

```
log_reg.score(X_train,y_train)
```

Output: 1

```
log_reg.score(X_test,y_test)
```

Output: 0.9722222222222222

Effect: A high C value of 10 applies a weak penalty, allowing the model to have more complexity.

Resulting feature selection: With L1 regularization, a low penalty will cause fewer coefficients to be driven to zero. This can result in a feature set that includes many irrelevant or noisy features that do not contribute meaningfully to the model's predictive power.

Consequence: The model may capture the noise in the training data, leading to overfitting. It will perform well on the training data but poorly on the test data.

